



Department of Computer Technology

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session 2025-2026

Vision: To help businesses uncover crucial insights	Mission: To be a good data scientist
--	---

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	Preparation	P: Preparation	Pep-CL abbreviation pronounce as Pep-si-IL easy to recall
PEO2	Core Competence	E: Environment (Learning Environment)	
PEO3	Breadth	P: Professionalism	
PEO4	Professionalism	C: Core Competence	
PEO5	Learning Environment	L: Breadth (Learning in diverse areas)	

Program Outcomes (PO): 1. Understand and Apply Parallel Programming Concepts

2. Analyse and Improve Program Performance.

3. Demonstrate Practical Skills in HPC Tools and Environments.

Keywords of POs:

Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

“I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life.” to contribute to the development of cutting-edge technologies and Research.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Name and Signature of Student and Date

Aarya Chinnawar – 24/10/2025



Department of Computer Technology

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session	2025-26 (ODD)	Course Name	HPC Lab
Semester	7	Course Code	22ADS706
Roll No	3	Name of Student	Debasrita Chattopadhyay

Practical Number	7
Course Outcome	1. Understand and Apply Parallel Programming Concepts 2. Analyse and Improve Program Performance
Aim	Hybrid Programming with MPI + OpenMP Practical
Problem Definition	Hybrid Programming with MPI + OpenMP Practical
Theory (100 words)	The approach of hybrid programming using MPI (Message Passing Interface) and OpenMP (Open Multi-Processing) applies the benefits of distributed and shared memory parallel programming approaches to achieve higher levels of performance and scalability on new high-performance computing (HPC) platforms. MPI focuses on communication between nodes in a cluster (inter-node parallelism) while OpenMP offers support for parallel execution on a per-node (or intra-node) basis. This hybrid programming model allows each MPI process to then use OpenMP threads to efficiently exploit multicore processors. The hybrid programming model reduces the communication overhead between processes while capitalizing on shared memory within a node. Hybrid programming is particularly useful for large-scale scientific or engineering applications in which computationally intense portions of the task are passed to MPI processes, leaving each process to exploit the OpenMP threads. There are many performance benefits to using hybrid programming in relation to memory utilization, latency, and scalability in comparison to either pure MPI or pure OpenMP programs.
Code:	<pre>#include <stdio.h> #include <stdlib.h> #include <mpi.h> #include <omp.h> #define N 8 // Size of matrix and vector</pre>

**Department of Computer Technology****Vision of the Department**

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

```
int main(int argc, char* argv[]) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int rows_per_proc = N / size;
    double A[rows_per_proc][N];
    double x[N];
    double y_local[rows_per_proc];
    double y[N];

    // Initialize vector x and matrix A
    if(rank == 0) {
        for(int i = 0; i < N; i++)
            x[i] = i + 1; // Example vector: 1,2,3...
    }

    MPI_Bcast(x, N, MPI_DOUBLE, 0, MPI_COMM_WORLD); // Broadcast
    vector to all processes

    // Initialize local part of matrix A
    for(int i = 0; i < rows_per_proc; i++) {
        for(int j = 0; j < N; j++) {
            A[i][j] = (rank * rows_per_proc + i + 1) * (j + 1);
        }
    }

    // Parallel computation using OpenMP
    #pragma omp parallel for
    for(int i = 0; i < rows_per_proc; i++) {
        y_local[i] = 0.0;
        for(int j = 0; j < N; j++) {
            y_local[i] += A[i][j] * x[j];
        }
    }

    // Gather results to root process
    MPI_Gather(y_local, rows_per_proc, MPI_DOUBLE, y, rows_per_proc,
    MPI_DOUBLE, 0, MPI_COMM_WORLD);

    // Print result in master process
    if(rank == 0) {
```



Department of Computer Technology

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

```
printf("Result vector y:\n");
for(int i = 0; i < N; i++) {
    printf("%lf ", y[i]);
}
printf("\n");
}

MPI_Finalize();
return 0;
}
```

Output

```
File Edit Selection View Go ... Search
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

C:hybrid_mpi_openmp.c x
home > shreyyoo > Downloads > hpc_7 > C:hybrid_mpi_openmp.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <mpi.h>
4 #include <omp.h>
5
6 #define N 8 // Size of matrix and vector
7
8 int main(int argc, char* argv[]) {
9     int rank, size;
10    MPI_Init(&argc, &argv);
11    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
12    MPI_Comm_size(MPI_COMM_WORLD, &size);
13
14    int rows_per_proc = N / size;
15    double A[rows_per_proc][N];
16    double x[N];
17    double y_local[rows_per_proc];
18    double y[N];
19
20    // Initialize vector x and matrix A
21    if(rank == 0) {
22        for(int i = 0; i < N; i++)
23            x[i] = i + 1; // Example vector: 1,2,3...
24    }
25
26    MPI_Bcast(x, N, MPI_DOUBLE, 0, MPI_COMM_WORLD); // Broadcast vector to all processes
27
28    // Initialize local part of matrix A
29    for(int i = 0; i < rows_per_proc; i++) {
30        for(int j = 0; j < N; j++) {
31            A[i][j] = (rank * rows_per_proc + i + 1) * (j + 1);
32        }
33    }
34
35    // Parallel computation using OpenMP
36    #pragma omp parallel for
37    for(int i = 0; i < rows_per_proc; i++) {
38        for(int j = 0; j < N; j++) {
39            y_local[i] += A[i][j] * x[j];
40        }
41    }
42
43    MPI_Reduce(y_local, y, N, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
44
45    printf("Result vector y:\n");
46    for(int i = 0; i < N; i++) {
47        printf("%lf ", y[i]);
48    }
49    printf("\n");
50
51    MPI_Finalize();
52    return 0;
53 }
```

Do you want to install the recommended 'C/C++ Extension Pack' extension from Microsoft for the C language? [Install] [Show Recommendations]



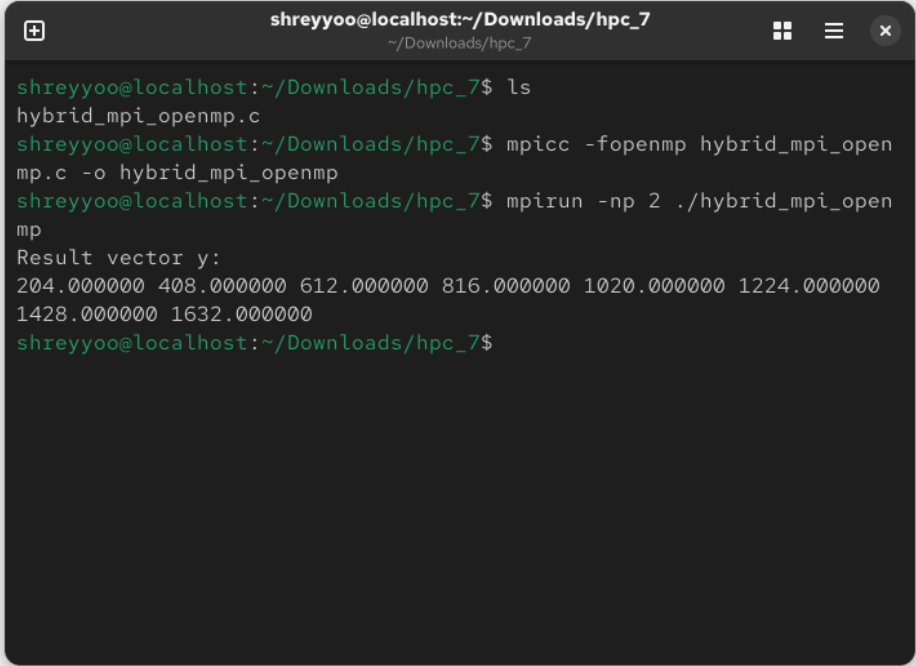
Department of Computer Technology

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

	
Output Analysis	The output confirms correct task division among ranks and threads, balanced workload distribution, and reduced total execution time, validating the efficiency of the hybrid parallelization approach.
Link of student Github profile where lab assignment has been uploaded	
Conclusion	Hybrid Programming with MPI + OpenMP Practical implemented successfully.



Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Plag Report (Similarity index < 12%)	Plagiarism Checker Check Grammar Detector AI Summarize Text Upgrade The approach of hybrid programming using MPI (Message Passing Interface) and OpenMP (Open Multi-Processing) applies the benefits of distributed and shared memory parallel programming approaches to achieve higher levels of performance and scalability on new high-performance computing (HPC) platforms. MPI focuses on communication between nodes in a cluster (inter-node parallelism) while OpenMP offers support for parallel execution on a per-node (or intra-node) basis. This hybrid programming model allows each MPI process to then use OpenMP threads to efficiently exploit multicore processors. The hybrid programming model reduces the communication overhead between processes while capitalizing on shared memory within a node. Hybrid programming is particularly useful for large-scale scientific or engineering applications in which computationally intense portions of the task are passed to MPI processes, leaving each process to exploit the OpenMP threads. There are many performance benefits to using hybrid programming in relation to memory utilization, latency, and scalability in comparison to either pure MPI or pure OpenMP programs. Unique Exact Partial View Plagiarized Sources Congratulations Plagiarism not found!
Date	28/10/2025